

ChamiNext × Apex Labs — Joint POC Workshop

Product Requirements Document | Uzziel Perez & Rich Lanot, Founders | June 2026 | *Draft — workshop discussion only* **Ask:**
Design partners + Bedrock pilot (credits, dedicated support, warm SMB intros) · Demo: [chaminext.netlify.app](#)

1 — Background

ChamiNext is a hiring and learning platform built for engineers who work natively alongside AI. The bet: the real differentiator in 2026 is **how well someone collaborates with AI** — prompting judgment, verification discipline, recovery when AI output is wrong — not whether they can write code unaided.

This workshop with Apex Labs is a pressure-test of that direction, and an exploration of AWS Bedrock as the infrastructure for an agentic evaluation layer. We want to leave with a scoped, shippable MVP — not a slide deck.

2 — The Four Evaluation Pillars

ChamiNext evaluates engineers across four dimensions that mirror real engineering work. Each maps to a distinct agent behavior and rubric.

Pillar	Description	Scope
Reasoning (Adaptive AI)	Socratic, reasoning-graded coding tasks. Tests decomposition and how candidates use AI to reason through constraints — signal, not recall. Maps to the deck's <i>AI Mocks</i> line.	MVP
Ship test	Build and deploy a scoped feature in a 24–72h window. Evaluates speed, code quality, and AI-augmented delivery judgment. Maps to the deck's <i>Ship Tests</i> line.	MVP
System design	Open-ended architecture discussion. Tests reasoning quality, trade-off awareness, and how candidates structure ambiguous problems with AI assistance.	Phase 2
AI collaboration vibe check	Live session scoring: prompt specificity, trust calibration, and recovery from bad AI output. ChamiNext's core differentiator.	Phase 2

MVP focus: Pillars 1 and 2 (reasoning + ship test) are the buildable POC target. They share an agent architecture — LLM-as-judge over a static submission — and have clear ground truth to validate against. This is deliberately a more validatable starting point than the deck's fully conversational AI Mock; the conversational/live experience (Pillars 3 and 4) requires live-session capture and a more nuanced rubric, scoped for Phase 2.

3 — MVP Architecture

A single evaluation pipeline handles both MVP pillars. Token usage is minimized by structuring evaluation as **one batched judge call per submission** — not a multi-turn agent loop.

Submission ingest → Rubric structuring → LLM-as-judge eval → Score + rationale

Component	Approach	AWS Service
Submission store	Repo / zip / PR link ingested per task window	S3
Rubric engine	Fixed rubric per task type; structured prompt with tool use for code-line citations	Bedrock (Claude 3.5 Sonnet)
Scoring consistency	Guardrails to bound outputs; repeated-run variance check on validation set	Bedrock Guardrails
Identity & score storage	Candidate identity and structured score/telemetry persistence (per deck pilot architecture)	Cognito + RDS
Human review handoff	Reviewer-facing view shows agent rationale alongside raw submission; flagged outliers queued for human check	TBD with Apex Labs

Component	Approach	AWS Service
Token budget	Single judge call per eval; no iterative agent loop in MVP. Rationale capped at ~300 tokens. Target €0.05–€0.30 per eval.	Bedrock

4 — MVP Scope

One task type per pillar. One fixed rubric. One labeled validation set to verify agent accuracy against human-grader scores.

In scope

- Algorithmic task (single problem, fixed rubric)
- Ship test task (1–2hr feature delivery)
- Score + per-dimension breakdown + cited rationale
- Reviewer-facing validation view
- Validation against labeled human-grader set

Out of scope

- Candidate-facing UI, scheduling, platform integration
- Multi-language / multi-task generalization
- Anti-plagiarism or cheating detection
- Live-session capture (Phase 2)
- System design or vibe-check pillars

5 — Open Questions for Apex Labs

- **Model selection** — Which Bedrock model gives the most consistent, auditable rubric scoring at this complexity? What does variance look like across repeated runs on the same submission?
- **Guardrails vs. custom harness** — Is Bedrock Guardrails the right fit for keeping scoring within rubric bounds and catching hallucinated code claims — or do we need a custom eval harness?
- **Rationale traceability** — What's the right pattern for citing specific code lines in rationale output — tool use, structured output, or post-processing?
- **Human-in-the-loop** — Where does human review sit in this pipeline? Does AWS have existing patterns or tooling for that handoff?
- **Cost per eval** — What does per-evaluation LLM cost look like on Bedrock at the token budget above, and how does it scale to hundreds of submissions per hiring cycle?

6 — Success Criteria

Workshop success

- Apex Labs gives concrete direction on Bedrock model and architecture fit
- Scoped POC agreed with clear next steps and rough timeline
- Clarity on credits or partnership path to support the build

POC success

- Agent scores correlate strongly with human-grader scores on validation set
- Rationale is specific enough a recruiter trusts it without re-reading the submission
- Pipeline runs end-to-end on Bedrock with acceptable latency and cost per eval

7 — Risks

- **Rubric subjectivity** — Code quality judgments vary even among human graders. The validation set must account for scoring variance, not assume a single correct score.
- **Candidate trust** — AI grading is only credible if the rationale is specific and traceable. Transparency is a product requirement, not a nice-to-have.
- **Scope creep** — Pillars 3 and 4 are more compelling narratively but materially harder. Be explicit in the workshop about what's POC-able in days vs. what's a longer build.

8 — Workshop Agenda

#	Topic	Time
1	ChamiNext context and product thesis	10 min
2	Four pillars walkthrough — MVP scope vs. Phase 2	15 min
3	Apex Labs technical reaction — architecture and AWS fit	20 min
4	Converge on POC scope and success criteria	15 min
5	Next steps, timeline, credits or partnership path	10 min

9 — Roadmap Beyond the POC

Context for Apex Labs on where ChamiNext is headed so they can see how the POC fits a larger system.

- **Skill-gap diagnostic agent** — turns assessment history into a personalized practice plan, closing the loop between hiring and the learning/practice side of the platform
- **Adversarial task generator** — generates fresh, hard-to-leak tasks per candidate, paired with the grading agent, to address assessment integrity in an AI-saturated hiring market
- **System design evaluation** — structured agent evaluates architecture discussions; requires a more nuanced rubric than code submission grading
- **AI collaboration vibe check** — live-session scoring is the long-term differentiator once the grading-agent pattern is proven

10 — Timeline & Sprints

This is the engineering core of the deck's **90-day collaborative Bedrock pilot** (slide 17): build in weeks 1–9, with the remainder of the 90 days for design-partner validation and the GTM co-sell motion. Plan: **6 weeks core build + 3 weeks buffer** (9 weeks total). Sprints are 1 week each, starting the week after the workshop. Buffer absorbs model-selection iteration, validation-set labeling delays, and AWS/credits setup.

Phase	Week	Sprint Goal	Key Deliverables	Owner
Sprint 0 — Workshop & Scope	Week 0	Align on architecture, model, and POC scope	Signed-off scope, Bedrock model pick, credits/partnership path, validation-set plan	Uzz + Apex Labs
Sprint 1 — Foundation	Week 1	Stand up infra and ingest	AWS account + Bedrock access, S3 submission store, repo/zip ingest, skeleton pipeline	Uzz
Sprint 2 — Reasoning judge (Pillar 1)	Week 2	First working eval on reasoning-graded coding task	Fixed rubric v1, single batched judge call, score + rationale output	Uzz
Sprint 3 — Ship-test judge (Pillar 2)	Week 3	Extend pipeline to ship-test task	Ship-test rubric, code-line citation in rationale, shared eval pipeline	Uzz
Sprint 4 — Validation & consistency	Week 4	Measure agent vs. human grader	Labeled validation set, repeated-run variance check, Guardrails tuning	Uzz + Apex Labs
Sprint 5 — Reviewer view & rationale	Week 5	Make output trustworthy & auditable	Reviewer-facing validation view, rationale traceability, outlier flagging	Uzz
Sprint 6 — Harden & cost	Week 6	End-to-end, measured, demo-ready	Cost-per-eval report, latency check, POC demo + write-up	Uzz

Phase	Week	Sprint Goal	Key Deliverables	Owner
Buffer	Weeks 7–9	Absorb slippage & iterate	Model re-tuning, rubric refinement, extra labeling, scope adjustments	—
Stretch (if ahead)	Weeks 7–9	Demo the deck's flagship	Single-turn conversational AI Mock prototype — Socratic prompt + rubric-scored response on Bedrock — to show the engine generalizes from static grading to the deck's headline experience	Uzz + Apex Labs

Milestones

- **M1 (end Week 1):** Pipeline ingests a submission end-to-end on Bedrock.
- **M2 (end Week 3):** Both MVP pillars produce scored, cited rationale.
- **M3 (end Week 4):** Agent scores correlate with human-grader set; variance characterized.
- **M4 (end Week 6):** Demo-ready POC with cost/latency numbers and validation results.

Dependencies / blockers to watch

- Bedrock access + credits confirmed before Sprint 1.
- Labeled human-grader validation set ready before Sprint 4 (start collecting in Sprint 0).
- Apex Labs availability for Sprints 0 and 4 (the two collaboration-heavy points).